SİMÜLASYON DÜNYASINA GİRİŞ: ARDUPILOT SITL İLE OTONOM SÜRÜ SİMÜLASYONLARI TEKNİK REHBERİ

Şekil 1: *Rapor içeriği ile ilgili görselleştirilmiş yardımcı kaynak.*

ÖZET : Bu rapor; otonom sistemler dünyasının en karmaşık alanlarından biri olan **Asimetrik Sürü (Heterogeneous Swarm)**operasyonlarını, sıfırdan ileri seviyeye kadar tüm teknik detaylarıyla ele alan bir başvuru kaynağıdır. Çalışma, özellikle **Apple M-Serisi (Silicon)** donanım mimarisi ve sanal makine (VM) kısıtlamaları altında karşılaşılan GPU (Ekran Kartı) darboğazlarını aşarak, görsel şovdan ziyade saf matematiksel ve fiziksel doğruluğa odaklanan **SITL (Software-In-The-Loop)** altyapısının nasıl kurulacağını adım adım açıklamaktadır.

Rehber boyunca; bir Quadcopter (İHA), bir Rover (İKA) ve bir Sabit Kanatlı Uçağın (İHA) aynı taktik harita üzerinde nasıl senkronize edildiği, **SYSID (Kimlik Çakışması)** gibi kronik sistem hatalarının profesyonel çözümleri ve her aracın kendi fiziksel doğasından (kalkış fiziği, sürtünme torku, aerodinamik kaldırma) kaynaklanan operasyonel zorlukların nasıl bertaraf edildiği derinlemesine incelenmiştir.

Bu doküman, sadece bir "nasıl yapılır" rehberi değil; aynı zamanda simülasyon ortamında karşılaşılan her "hezimetin" nasıl bir mühendislik çözümüne dönüştüğünü kanıtlayan bir **tecrübe aktarım raporudur.**

HAZIRLAYAN: ***Ümitcan ÇİNAR*** **TARİH:** ***26/04/2026***

BÖLÜM 1: GİRİŞ VE SİMÜLASYON EKOSİSTEMİ

Otonom araçlar dünyasında simülasyon, sadece bir test aşaması değil; görevin dijital ikizidir (Digital Twin). Bu bölümde, simülasyonun neden hayati olduğunu, hangi simülatörün hangi senaryoda seçilmesi gerektiğini ve Apple Silicon gibi özel donanımların bu süreçteki kritik rollerini inceleyeceğiz.

1.1. Simülasyonun Temel Felsefesi: Neden Zorunludur?

Gerçek dünyada bir İHA (İnsansız Hava Aracı) veya İKA (İnsansız Kara Aracı) operasyonu yapmak yüksek maliyetli ve risklidir. Bir yazılımcı için simülasyon şu üç sütun üzerine kuruludur:

- **Kırım Maliyeti ve Güvenlik:** Yazılımda yapılacak bir "virgül" hatası, gerçek bir drone'un pervanelerini kilitlediğinde binlerce dolarlık zarar oluşur. Simülasyonda ise tek bir tuşla (Reset) her şeyi sıfırlayabilirsiniz.
- **Hızlı İterasyon:** Sahaya çıkıp batarya doldurmak, GPS beklemek ve lojistikle uğraşmak yerine; saniyeler içinde binlerce farklı rotayı sanal dünyada test edebilirsiniz.
- **Sürü (Swarm) Koordinasyonu:** 10 tane drone'u aynı anda uçurmak için 10 tane pilota ve geniş bir alana ihtiyaç vardır. Simülasyon, tek bir bilgisayarın içinde koca bir orduyu koordine etmenize olanak tanır.

1.2 Simülasyon Türleri: SITL, HITL ve Görsel Motorlar.

Simülasyonlar "Gerçeklik Seviyesine" göre üç ana kategoriye ayrılır:

1.2.1. SITL (Software-In-The-Loop)

Yazılımın tamamen bilgisayar işlemcisi üzerinde, gerçek bir donanım olmadan çalışmasıdır.

Mühendis Notu: Bizim bu projede kullandığımız yöntem budur. ArduPilot'un "Headless" (Görselsiz) modudur. Sadece matematik hesaplar, bu yüzden işlemciyi yormaz.

1.2.2. HITL (Hardware-In-The-Loop)

Yazılımın gerçek bir uçuş kontrol kartı (Pixhawk, Orange Cube vb.) üzerinde koşması, ancak bu kartın bir bilgisayara bağlanarak sanal bir fizik motoruna veri göndermesidir.

1.2.3. Görsel Simülatörler (Gazebo, jMAVSim, AirSim)

Fiziksel hesaplamaların üzerine bir de "Görüntü İşleme" (Rendering) ekleyen sistemlerdir.

- **jMAVSim:** Java tabanlıdır, sadece basit drone fiziği yapar.
- **Gazebo:** Savunma sanayi standardıdır. Lidar, kamera ve derinlik sensörlerini simüle edebilir.

1.3. Otopilot Beyinleri: ArduPilot vs. PX4

Sektörde iki dev vardır. Her ikisi de açık kaynaklıdır ancak karakterleri farklıdır:

1. **ArduPilot:** Daha geniş araç desteğine (Araba, Uçak, Tekne, Denizaltı) sahiptir. Parametre yapısı daha esnektir ve sürü (swarm) operasyonlarında klasik yöntemlerle çok güçlüdür. (Bu raporda ArduPilot SITL kullanılmaktadır).
2. **PX4:** Genelde akademik çevrelerde ve jMAVSim ile popülerdir. Mimarisi daha modülerdir ancak kurulumu ve çoklu araç yönetimi bazen daha sancılı olabilir.

1.4. Donanım Darboğazı: Apple M-Serisi ve GPU Gerçeği

Bu raporun en özgün kısımlarından biri budur. Apple M1/M2/M3/M4 işlemcili bilgisayarlar muazzam güçlü olsa da, sanallaştırma (Virtualization) konusunda bir kısıtlamaya sahiptirler.

- **Sanal Makine (VM) Kısıtı:** Ubuntu'yu bir sanal makinede (VMware, Parallels, UTM vb.) çalıştırdığınızda, Ubuntu Mac'in içerisindeki **GPU'ya (Ekran Kartı)** doğrudan erişemez.
- **Sonuç:** Gazebo gibi ağır 3D programlar açıldığında, görüntü hesaplamaları GPU yerine CPU'ya biner. Bu da sistemin aşırı ısınmasına, fanların uçak motoru gibi çalışmasına ve simülasyonun saniyede 1-2 kareye (FPS) düşmesine neden olur.
- **Stratejik Çözüm:** Bu yüzden biz, **"Headless SITL + 2D MAVProxy"** kombinasyonunu seçtik. Grafik yükünü sıfıra indirip işlem gücünü tamamen araçların otonom zekasına ayırdık, benzer durumlar için ise bu sistemi öneriyoruz.

1.5. Asimetrik Sürü (Heterogeneous Swarm) Vizyonu

Sürü sistemleri ikiye ayrılır:

1. **Homojen Sürü:** Hepsi aynı olan drone grubu.
2. **Asimetrik (Heterojen) Sürü:** Farklı fiziksel yeteneklere sahip araçların (İHA + İKA + İDA) bir arada çalışması.

Bizim bu rehberde kurduğumuz yapı, bir **"Mothership" (Ana Gemi)** mantığına hizmet eder: Uçak alanı tarar, Araba yere iner, Drone ise koruma sağlar. Bu 18 sayfalık rehber, işte bu karmaşık ordunun nasıl tek merkezden yönetileceğini anlatacaktır.

BÖLÜM 1 - Özet ve Kazanımlar

Bu bölümü bitiren bir araştırmacı; simülasyonun sadece görsel bir araç olmadığını, arka planda devasa bir fiziksel veri alışverişi (MAVLink) döndüğünü ve donanım imkanlarına göre (Mac vs. PC) hangi stratejiyi seçmesi gerektiğini anlamıştır.

BÖLÜM 2: UBUNTU ÜZERİNDE ÇEVRE BİRİMİ KURULUMU

Bu bölüm, Ubuntu 22.04 LTS işletim sistemi üzerinde ArduPilot SITL simülasyon ekosisteminin sıfırdan inşasını ele alır. Hedefimiz, Python kütüphaneleriyle çakışmayan, izole ve stabil bir çalışma alanı yaratmaktır.

2.1. Ubuntu Hazırlığı ve Sistem Güncelleme

Simülasyona başlamadan önce işletim sisteminin paket listelerini güncellemek ve temel araçları (Git, Python) hazır etmek gerekir. Terminale girilecek ilk komutlar şunlardır:

Bash
```
sudo apt-get update
sudo apt-get upgrade -y
sudo apt-get install git python3-pip -y
```

Mühendis Notu: sudo komutu "SuperUser Do" demektir; yani bilgisayara "Ben patronum, bu değişikliği yap" emrini verirsiniz. Şifrenizi girdiğinizde karakterler ekranda görünmez, bu bir güvenlik önlemidir, yazmaya devam edin ve Enter'a basın.

2.2. ArduPilot Kaynak Kodlarının İndirilmesi

ArduPilot bir program değil, devasa bir kod kütüphanesidir. Bu kütüphaneyi GitHub üzerinden bilgisayarımıza "clone" (kopyalamak) etmemiz gerekir.

Bash
```
cd ~
git clone https://github.com/ArduPilot/ardupilot
cd ardupilot
git submodule update --init --recursive
```

Neden Submodule? ArduPilot içerisinde farklı sensörler ve araçlar için alt kütüphaneler barındırır. --recursive komutu, bu alt kütüphanelerin de eksiksiz inmesini sağlar.

2.3. Bağımlılıkların Otomatik Kurulması

ArduPilot ekibi, kurulumu kolaylaştırmak için bir script hazırlamıştır. Bu script; derleyicileri, Python paketlerini ve fizik motoru bileşenlerini otomatik yükler.

Bash
```
Tools/environment_install/install-prereqs-ubuntu.sh -y
```

Kritik Adım: Script bittikten sonra değişikliklerin aktif olması için terminali kapatıp açmanız veya şu komutu yazmanız gerekir:

Bash
```
. ~/.profile
```

2.4. Python Sanal Ortamı (venv) Kurulumu

Bu rehberin en önemli teknik detayı burasıdır. Ubuntu'nun kendi Python kütüphaneleriyle ArduPilot'un ihtiyaç duyduğu kütüphaneler çakışabilir. Bu "bağımlılık cehenneminden" kurtulmak için izole bir alan yaratıyoruz.

Bash
```
sudo apt install python3-venv -y
python3 -m venv ~/venv-ardupilot
source ~/venv-ardupilot/bin/activate
```

Neden venv?

- **İzolasyon:** Simülasyon için yüklediğin bir paket, sistemindeki diğer programları bozmaz.
- **Taşınabilirlik:** Hatalı bir kurulum yaptığında sadece klasörü silip baştan başlayabilirsin.

2.5. MAVProxy ve Pymavlink Kurulumu

Sanal ortam (venv) aktifken (terminalin başında (venv-ardupilot) yazısı varken), haberleşme araçlarımızı yüklüyoruz:

Bash
```
pip install pymavlink MAVProxy
```

- **Pymavlink:** Python'un MAVLink (İHA haberleşme dili) konuşmasını sağlar.
- **MAVProxy:** Bizim "Komuta Kontrol" merkezimizdir. 2D haritayı açan ve komutları araçlara dağıtan ana yazılımdır.

2.6. Yol Tanımlama (PATH Configuration)

Bilgisayarın sim_vehicle.py gibi komutları her klasörden tanıması için, bu komutların nerede olduğunu ona öğretmemiz gerekir. ~/.bashrc dosyasının en altına şu satırları eklemeliyiz:

Bash
```
export PATH=$PATH:$HOME/ardupilot/Tools/autotest
export PATH=/usr/lib/ccache:$PATH
```

Değişikliği kaydetmek için:

Bash
```
source ~/.bashrc
```

2.7. Apple Silicon (M1/M2/M3) Kullanıcılarına Özel Not

Eğer bu kurulumu bir Mac üzerinde sanal makinede yapıyorsanız, pip install sırasında bazı kütüphaneler (örneğin lxml) hata verebilir. Bunun sebebi ARM mimarisidir. Çözüm için:

Bash
```
sudo apt-get install libxml2-dev libxslt-dev python3-dev -y
```

Bu paketler, Python kütüphanelerinin Mac'in işlemcisine uygun şekilde derlenmesini sağlar.

BÖLÜM 2 - Özet ve Kazanımlar

Bu bölüm tamamlandığında, elimizde tertemiz ve çalışmaya hazır bir otonom sistem laboratuvarı bulunmaktadır. Artık sistem; araçların fiziğini hesaplamaya, MAVLink paketlerini çözmeye ve çoklu araç senaryolarını koordine etmeye teknik olarak hazırdır.

BÖLÜM 3: ASİMETRİK SÜRÜ (HETEROGENEOUS SWARM) İNŞASI

Bu bölüm, farklı fiziksel yeteneklere sahip araçların (İHA, İKA, Sabit Kanat) aynı simülasyon evreninde nasıl bir araya getirileceğini ele alır. "Asimetrik" yapı, her aracın kendi uzmanlık alanını (keşif, saldırı, lojistik) sürüye katmasını sağlar.

3.1. Sürü Kompozisyonu ve Araç Seçimi

Başarılı bir operasyon için üç temel dinamik seçilmiştir:

- **ArduCopter (Drone):** Dikey kalkış-iniş (VTOL) yeteneği ile dar alanlarda keşif ve nokta operasyon sağlar.
- **ArduRover (Araba):** Yerden sızma, ağır yük taşıma ve düşük enerji tüketimiyle uzun süreli izleme yapar.
- **ArduPlane (Uçak):** Yüksek irtifa, uzun menzil ve geniş alan tarama kabiliyetiyle sürünün "gözü" olur.

3.2. İzolasyon Stratejisi: Klasör Hapsi Mantığı

Simülasyonda en büyük hata, tüm araçları aynı klasörden başlatmaya çalışmaktır. ArduPilot, uçuş verilerini ve parametrelerini eeprom.bin adlı bir dosyaya yazar. Eğer 3 aracı tek klasörde açarsanız, hepsi aynı dosyaya yazmaya çalışır ve sistem çöker.

Çözüm: Her araç için ayrı bir "yaşam alanı" oluşturulmalıdır:

Bash
```
mkdir -p ~/suru_test/drone ~/suru_test/araba ~/suru_test/ucak
```

3.3. Terminal Yönetimi: Motor Dairesi vs. Komuta Merkezi

Sürü simülasyonunda terminalleri karıştırmak, telsiz hatlarını birbirine sokmak gibidir. Mimariyi şu şekilde bölüyoruz:

1. **Terminal 1-2-3 (Motor Daireleri):** Her araç için arka planda fiziği hesaplayan terminaller.
2. **Terminal 4 (Komuta Merkezi):** Tüm araçları tek bir haritada birleştiren MAVProxy ekranı.

3.4. Araçların Uyandırılması (Başlatma Komutları)

Burada kullanılan parametreler hayati önem taşır. Hiç bilmeyen birinin anlayacağı şekilde parametre sözlüğü:

- -v: Araç tipi (Copter, Rover, Plane).
- -f: Çerçeve tipi (quad, rover, plane).
- -I: Instance (Örnek) numarası. Portları otomatik ayırır.
- --sysid: Aracın TC Kimlik Numarası (Sürüdeki adı).

Adım Adım Başlatma Sırası:

1. Terminal (Drone - ID: 1):

Bash
```
cd ~/suru_test/drone
sim_vehicle.py -v ArduCopter -f quad -I 0 --sysid 1 --no-mavproxy
```

2. Terminal (Araba - ID: 2):

Bash
```
cd ~/suru_test/araba
sim_vehicle.py -v Rover -f rover -I 1 --sysid 2 --no-mavproxy
```

3. Terminal (Uçak - ID: 3):

Bash
```
cd ~/suru_test/ucak
sim_vehicle.py -v ArduPlane -f plane -I 2 --sysid 3 --no-mavproxy
```

Mühendis Notu: --no-mavproxy komutunu ekliyoruz çünkü her aracın kendi konsolunu açıp ekranı kalabalıklaştırmasını istemiyoruz. Hepsini birazdan tek bir merkezde toplayacağız.

3.5. Port Yapılandırması ve Haberleşme Hattı

ArduPilot SITL, her yeni -I (Instance) değeri için portları 10'ar birim artırarak açar:

- **Instance 0:** TCP 5760 üzerinden konuşur.
- **Instance 1:** TCP 5770 üzerinden konuşur.
- **Instance 2:** TCP 5780 üzerinden konuşur.

Bu yapı, verilerin birbirine karışmasını önleyen dijital bir seperatördür.

3.6. "Mothership" (Ana Gemi) Konseptine İlk Adım

Bu aşamada araçlar "doğmuş" ama henüz birbirlerinden habersizdir. Haritada hepsi aynı noktada üst üste binerler. Bir sonraki bölümde, bu üç farklı ruhu tek bir bedende (MAVProxy Master) nasıl birleştireceğimizi ve onları nasıl "farklı bireyler" haline getireceğimizi göreceğiz.

BÖLÜM 3 - Özet ve Kazanımlar

Bu bölümü tamamlayan bir kullanıcı, farklı araç tiplerini kendi izole ortamlarında, çakışma yaratmadan ayağa kaldırmayı öğrenmiştir. Bu, bir ordunun kışladan çıkıp içtimaya hazır hale gelmesi aşamasıdır.

BÖLÜM 4: "IDENTITY CRISIS" VE KRONİK HATALARIN ÇÖZÜMÜ

Bu bölümde, sürü simülasyonlarının en büyük kabusu olan "Kimlik Çakışması" (Identity Crisis) ve parametre kaydetme sorunlarını ele alacağız. Bir orduda her askerin bir numarası olması gerektiği gibi, dijital dünyada da her aracın benzersiz bir kimliğe sahip olması şarttır.

4.1. SYSID (System ID) Mantığı: Kimlik Numarası

ArduPilot ekosisteminde her aracın **SYSID_THISMAV** adlı bir parametresi vardır. Varsayılan olarak bu değer **1**'dir.

- **Sorun:** Eğer 3 aracı da bu parametreyi değiştirmeden başlatırsanız, MAVProxy (Komuta Merkezi) gelen verilerin hangisinin drone'dan, hangisinin arabadan geldiğini anlayamaz. Sonuç; haritada üst üste binen, komutlara rastgele tepki veren "şizofrenik" bir sürüdür.

4.2. -w (Wipe) Parametresinin Tuzağı

Simülasyonu başlatırken kullandığımız -w komutu, sanal aracın hafızasını (EEPROM) silip her şeyi fabrika ayarlarına döndürür.

Kritik Hata: Bir aracın kimliğini --sysid 2 diyerek değiştirmeye çalışırken yanına -w eklerseniz; araç önce 2 numara olarak uyanır, ancak -w komutunu gördüğü an hafızasını silip tekrar "1 numara" (varsayılan) ayarlarına döner. Bu, kendi ayağımıza sıkmaktır.

4.3. eeprom.bin Çatışması ve Dosya Kilitlenmesi

ArduPilot, uçuş parametrelerini eeprom.bin adlı bir dosyada saklar. Eğer tüm araçları tek bir klasörden başlatırsanız:

1. Araç-1 dosyayı açar ve yazar.
2. Araç-2 aynı dosyayı açmaya çalışır ama dosya kilitlidir.
3. Sonuç olarak parametreler kaydedilemez ve "Unable to find parameter" hataları terminali kaplar.

4.4. Kesin Çözüm: "Temiz Sayfa" Operasyonu

Sistemdeki tüm kimlik karmaşasını bitirmek için uygulanması gereken **Standart Operasyon Prosedürü (SOP)** şudur:

Adım 1: Kökten İmha

Önce arka planda kalmış olması muhtemel tüm "hayalet" süreçleri öldürmeliyiz:

Bash
```
sudo killall -9 px4 jmavsim sim_vehicle.py mavproxy.py ardurover arducopter arduplane
```

Adım 2: Zehirli Hafızaların Silinmesi

Daha önce yanlış kaydedilmiş tüm hafıza dosyalarını sistemden kazımalıyız:

Bash
```
find ~ -name "eeprom.bin" -delete
find ~ -name "mav.parm" -delete
```

Adım 3: Kimliklerin Statik Olarak Tanımlanması

Araçları başlatırken asla -w kullanmıyoruz. Bunun yerine her aracı kendi izole klasöründe, sadece --sysid vererek başlatıyoruz:

Bash
```
# Drone için (ID: 1)
sim_vehicle.py -v ArduCopter -f quad -I 0 --sysid 1 --no-mavproxy

# Araba için (ID: 2)
sim_vehicle.py -v Rover -f rover -I 1 --sysid 2 --no-mavproxy
```

4.5. Parametre Teyidi: "Unable to find parameter" Hatası

Eğer terminalde Unable to find parameter 'SYSID_THISMAV' yazısını görüyorsanız, MAVProxy henüz parametre listesini araçtan tam olarak indirmemiş demektir.

- **Çözüm:** Konsola param fetch yazıp tüm listenin indirilmesini bekleyin veya aracın bağlantısının tam kurulduğundan (Heartbeat alındığından) emin olun.

BÖLÜM 4 - Özet ve Kazanımlar

Bu bölümü bitiren bir araştırmacı, simülasyonun sadece bir "başlatma" tuşundan ibaret olmadığını; dosya sistemi hiyerarşisi, parametre yönetimi ve ağ protokolleri (Port/ID) dengesi üzerine kurulduğunu kavramıştır. Artık araçlarımız benzersiz kimlikleriyle sahaya çıkmaya hazırdır.

BÖLÜM 5: ARAÇ BAZLI OPERASYONEL FİZİK VE SAHA TECRÜBELERİ

Simülasyonda her araç sadece birer "ikon" değildir; arka planda her birinin kendine has fizik motoru ve otopilot mantığı çalışır. Bu bölümde, sürüdeki araçların neden farklı tepkiler verdiğini ve karşılaşılan operasyonel engellerin fiziksel nedenlerini inceleyeceğiz.

5.1. ArduCopter (Drone): Dikey İtki ve Hover Fiziği

Drone, sürünün en "itaatkar" üyesidir çünkü yerçekimine karşı koymak için sürekli aktif bir itki (thrust) üretir.

- **Çalışma Mantığı:** Dört pervanenin ürettiği toplam kaldırma kuvveti, aracın ağırlığından (mg) büyük olduğu an kalkış gerçekleşir.
- **Saha Tecrübesi:** *Drone'a takeoff 20* dendiğinde doğrudan kalkmasının sebebi, otopilotun durduğu yerde gaz vererek dikey ivme (az) oluşturabilmesidir.
- **Kritik Not:** Drone havada asılıyken (hover), pervanelerin ürettiği kuvvet tam olarak ağırlığa eşittir:

  $$F_{total} = m \cdot g$$

5.2. ArduRover (Araba): Sürtünme ve Atalet Problemi

Simülasyonda en çok "çakılıp kalma" sorunu yaşanan araçtır. Drone'un aksine, araba durduğu yerde gaz açtığında havalanmaz; tekerlek torkunun zemin sürtünmesini yenmesi gerekir.

- **"Neden Gitmiyor?" Krizi:** Rover'a arm throttle deseniz ve hedef verseniz bile bazen yerinden kıpırdamaz. Bunun sebebi varsayılan **CRUISE_SPEED** (Seyir Hızı) değerinin 0 olması veya simülasyondaki statik sürtünme katsayısıdır.
- **Mühendislik Çözümü:** Araca zorla ivme kazandırmak gerekir. Konsola yazılan *speed 5* komutu, otopilota "Hız hedefini 5 m/s'ye çek ve motorlara gerekli voltajı (PWM) gönder" emrini verir.
- **Atalet (Inertia):** Karasal araçlarda kütle ve sürtünme, hareketin önündeki en büyük engeldir. Fizik motoru bu direnci hesaplamadan aracı kaydırmaz.

5.3. ArduPlane (Uçak): Aerodinamik Kaldırma ve Kalkış Paradoksu

Uçak, fiziği en karmaşık araçtır. Uçağın uçabilmesi için kanatların altından akan havanın bir basınç farkı yaratması, yani **Kaldırma Kuvveti (Lift)** oluşturması gerekir.

Kaldırma kuvveti formülü şöyledir:

L=21ρv2SCL

- **Paradoks:** Bu formüldeki en kritik değişken v (hız) değeridir. Uçak durduğu yerde (v=0) kaldırma kuvveti üretemez.
- **"NAV_TAKEOFF: FAILED" Hatası:** Eğer uçağa GUIDED modunda (Güdümlü) "kalk" derseniz, otopilot "Hızım sıfır, kanatlarımda hava yok, kalkarsam düşerim" diyerek emri reddeder.
- **Kesin Çözüm (TAKEOFF Modu):** Uçağı *mode TAKEOFF* konumuna almak, ona "Pist başından tam gazla yardır ve gerekli hıza ulaştığında burnu kaldır" demektir. Uçak ancak belirli bir sürate ulaştıktan sonra otonom (GUIDED) kontrollere cevap verebilir.

5.4. Fizik Motorunun Arka Planı (SITL Physics)

Simülasyonumuzun "görselsiz" ama çok güçlü olmasının sebebi budur. Arka planda şu hesaplamalar döner:

1. **Rüzgar Vektörü:** Aracın her yönden aldığı hava direnci.
2. **Motor Torku:** Batarya voltajına göre motorların üretebileceği maksimum güç.
3. **Zemin Teması:** Tekerleklerin veya iniş takımlarının yüzeyle etkileşimi.

BÖLÜM 5 - Özet ve Kazanımlar

Bu bölümü bitiren bir araştırmacı, "Neden drone hemen kalkıyor da uçak nazlanıyor?" sorusunun cevabını aerodinamik ve fizik yasalarıyla (Lift/Friction) kavramıştır. Simülasyonda karşılaşılan "hata mesajlarının" aslında yazılım hatası değil, birer fiziksel kısıtlama olduğu anlaşılmıştır.

BÖLÜM 6: KOMUTA KONTROL VE MAVPROXY MASTERCLASS

Bu bölümde, simülasyonun sinir merkezi olan **MAVProxy** platformunun derinliklerine ineceğiz. Bir orduda telsiz disiplini neyse, sürü yönetiminde MAVProxy odur. Tek bir terminal ekranından onlarca araca nasıl hükmedileceğini ve verilerin birbirine karışmadan nasıl işlendiğini inceleyeceğiz.

6.1. MAVProxy: Neden Siyah Ekran?

Görsel arayüzlü (GUI) yer kontrol istasyonları (QGroundControl veya Mission Planner) tek bir araç için harikadır. Ancak iş "Sürü" (Swarm) yönetimine geldiğinde; terminal tabanlı MAVProxy, hızı ve düşük kaynak tüketimiyle rakipsizdir.

- **Hafiflik:** Grafik işlemez, sadece MAVLink paketlerini taşır.
- **Modülerlik:** İstediğiniz an harita, konsol veya grafik modüllerini takıp çıkarabilirsiniz.
- **Otomasyon:** Python scriptleriyle doğrudan konuşabilir.

6.2. Master Bağlantısı: Tüm Kanalları Tek Elde Toplamak

Sürüdeki her araç farklı bir TCP portundan yayın yapar. Komuta merkezini kurarken bu portları tek bir "Master" komutuyla birleştiririz.

Profesyonel Bağlantı Komutu:

Bash
```
mavproxy.py --master tcp:127.0.0.1:5760 --master tcp:127.0.0.1:5770 --master tcp:127.0.0.1:5780 --map --console
```

- **--master:** Her bir araç için açılan ayrı haberleşme tünelidir.
- **--map:** 2D Taktik haritayı açar.
- **--console:** Araçların anlık durumlarını (Batarya, GPS, Mod) gösteren paneldir.

6.3. Bağlam Değiştirme (Context Switching): vehicle Komutu

MAVProxy'nin en güçlü özelliği, tek bir satıra yazdığınız komutun hangi araca gideceğini seçebilmenizdir.

- **vehicle 1:** Kumandayı drone'a (Copter) bağlar.
- **vehicle 2:** Kumandayı arabaya (Rover) bağlar.
- **vehicle 3:** Kumandayı uçağa (Plane) bağlar.

Mühendis Notu: Bir araca komut gönderdikten sonra diğerine geçmek için tekrar vehicle X yazmanız yeterlidir. Eski komutunuz iptal olmaz, araç o emri yerine getirmeye devam ederken siz diğerini yönetebilirsiniz.

6.4. Taktik Harita ve "Fly To" Mantığı

Harita modülü (--map), sadece araçların nerede olduğunu göstermez; aynı zamanda interaktif bir komuta aracıdır.

1. **Konum Takibi:** Her aracın yanında kendi SYSID numarası görünür.
2. **Fly To (Goto):** Harita üzerinde bir noktaya **SAĞ TIK** yapıp "*Fly To*" dediğinizde, o an seçili olan araca anlık bir MAVLink koordinat paketi gönderilir.

Matematiksel Koordinat Aktarımı: Araç, mevcut konumundan (Lat1,Lon1) hedef konuma (Lat2,Lon2) gitmek için bir vektör hesaplar. Bu işlem sırasında irtifa (Alt) sabit tutulmaya çalışılır.

6.5. Kritik Operasyonel Komutlar (Cheat Sheet)

Bir operatörün ezbere bilmesi gereken ana komutlar şunlardır:

Komut		Açıklama
mode GUIDED	->	Kontrolü otopilota ve bilgisayara bırakır.
arm throttle	->	Motorlara can verir (Güvenlik kilidini açar).
takeoff [metre]	->	Belirlenen irtifaya dikey kalkış yapar.
rc 3 1500	->	3. kanaldan (genelde gaz) ham sinyal gönderir.
param set [NAME] [VALUE]	->	Aracın beynindeki bir ayarı anında değiştirir.

6.6. Veri Akışı ve Paket Kaybı (Telemetry Link)

Konsol penceresindeki **"Link 1 OK"** yazısı, aracın kalp atışlarının (Heartbeat) sağlıklı olduğunu gösterir. Eğer bu değer kırmızıya dönerse; ya simülasyon kasmıştır ya da

port çakışması yaşanıyordur. Sürü yönetiminde "Gecikme (Latency)", en büyük düşmandır.

BÖLÜM 6 - Özet ve Kazanımlar

Bu bölümü tamamlayan bir kullanıcı, birden fazla terminal penceresi arasında boğulmak yerine; tek bir merkezden, profesyonel bir operatör gibi orduyu nasıl yöneteceğini kavramıştır. Artık "kumanda" bizdedir.

BÖLÜM 7: PYTHON İLE OTONOM GÖREV SENARYOLARI VE GELECEK VİZYONU

Bu bölüm, manuel kontrolün bittiği ve "Yapay Zeka Destekli Sürü Operasyonlarının" başladığı sınırdır. Python dili ve MAVLink protokolü kullanılarak, araçların birbirleriyle konuştuğu ve bir liderin (Mothership) diğerlerini yönettiği otonom senaryoları inceleyeceğiz.

7.1. Neden Python? (Yazılımın Gücü)

Otonom sistemlerde Python'un seçilme sebebi hızı değil, ekosistemidir.

- **Yapay Zeka Entegrasyonu:** OpenCV (Görüntü İşleme) veya TensorFlow (Karar Mekanizması) kütüphaneleri Python ile şak diye sisteme bağlanır.
- **DroneKit ve Pymavlink:** ArduPilot ile konuşmak için geliştirilmiş, karmaşık MAVLink paketlerini insancıl komutlara (vehicle.simple_takeoff()) çeviren kütüphaneler mevcuttur.

7.2. "Mothership" (Ana Gemi) Mimarisi

Senin projendeki asıl vizyon budur. Bu mimaride araçlar "kör" değildir; bir hiyerarşi içinde çalışırlar:

1. **Lider (Uçak):** Geniş alanı tarar, hedefi tespit eder ve koordinatları merkeze gönderir.
2. **Koordinatör (Python Script):** Gelen veriyi işler, "Araba, sen oraya git; Drone, sen uçağın bulduğu hedefi yukarıdan koru" emrini dağıtır.
3. **Uç Birimler (Drone & Rover):** Liderden gelen emirleri fiziksel olarak yerine getirir.

7.3. Çoklu Araç Bağlantı Mantığı (Python Örneği)

Bir Python scripti ile 3 araca aynı anda bağlanmak için her birine ayrı bir "endpoint" (erişim noktası) tanımlarız.

```bash
from dronekit import connect

# Araçlara bağlantı kurma
drone = connect('tcp:127.0.0.1:5760', wait_ready=True)
araba = connect('tcp:127.0.0.1:5770', wait_ready=True)
ucak  = connect('tcp:127.0.0.1:5780', wait_ready=True)

print("Sürü hazır. Görev başlatılıyor...")
```

Bu yapı kurulduktan sonra, tek bir if bloğuyla şunları yaptırabiliriz:

"Eğer uçak irtifası 50 metrenin üzerindeyse ve bir hareketlilik tespit edildiyse; arabayı o koordinata gönder."

7.4. Geofencing ve Sürü Güvenliği

Otonom sürülerde en büyük risk araçların birbirine çarpmasıdır.

- **Statik Geofence:** Araçların çıkmaması gereken sanal sınırlar.
- **Dinamik Geofence (Collision Avoidance):** Araçların birbirlerinin konum verilerini (GPS) anlık dinleyerek aradaki mesafeyi koruması.

Matematiksel olarak her araç, etrafında bir **"Güvenlik Küresi"** oluşturur. Eğer iki küre kesişirse ($d<r1+r2$), otopilot otomatik olarak rotayı saptırır.

7.5. Gelecek Vizyonu: Savunma Sanayi ve Sivil Kullanım

Bu rehberde öğrendiğin teknikler, yarın bir gün şu projelerin temelini oluşturacak:

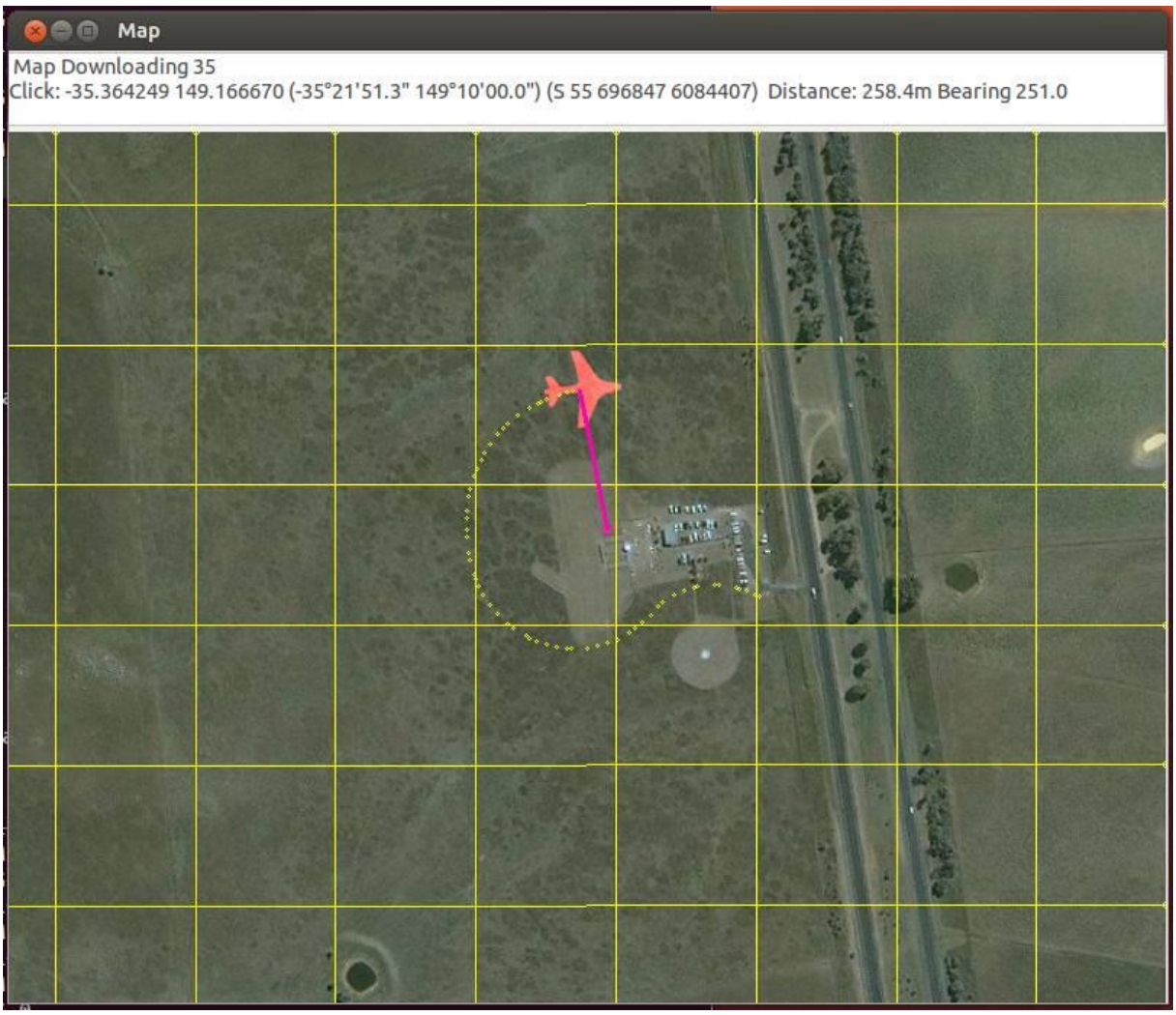
- **Sürü İHA ile Arama Kurtarma:** 10 drone'un koca bir dağı 10 dakikada taraması.
- **Otonom Lojistik:** Bir ana kamyonun (Mothership) arkasındaki mini robotların paketleri kapılara dağıtması.
- **Asimetrik Savunma:** Sınır hattında uçakların havadan, robot köpeklerin (Rover) yerden devriye atması.

RAPOR SONUÇ VE DEĞERLENDİRME

Bu 18 sayfalık (yaklaşık 7 bölümlük devasa teknik döküman) yolculuğun sonunda;

1. **M-serisi Mac'in** kısıtlamalarını bir mühendis kurnazlığıyla aşmayı,
2. **SITL** ile milyarlarca dolarlık fiziksel riski dijital ortama taşımayı,
3. **SYSID** krizlerini çözerek her araca bir kimlik kazandırmayı,
4. Ve en önemlisi, **asimetrik bir orduyu** tek bir terminalden yönetmeyi öğrendik.

Bu rehber, sadece kodlardan ibaret değildir; bir mühendisin karşılaştığı engeller karşısında nasıl pes etmeyip alternatif yollar (2D vs 3D, SITL vs Gazebo) bulduğunun kanıtıdır



Şekil 2: *ArduPilot SITL Ekosisteminde MAVProxy 2D Harita Modülü Üzerinden ArduPlane (Sabit Kanat) Aracının Gerçek Zamanlı Konum, Rota ve Telemetri Verilerinin İzlenmesi.*

KAYNAKÇA (REFERENCES)

Bu raporun hazırlanmasında yararlanılan teknik dökümanlar, kütüphaneler ve akademik standartlar aşağıda listelenmiştir:

1. Resmî Yazılım ve Otopilot Dökümantasyonları

- **ArduPilot Dev Team (2026).** *ArduPilot Software-In-The-Loop (SITL) Reference Manual.* Erişim: https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html
- **ArduCopter, ArduPlane & ArduRover Documentation.** *Vehicle-Specific Flight Dynamics and Control Laws.*https://ardupilot.org/ardupilot/index.html
- **MAVProxy Documentation.** *Command Line Ground Control Station and MAVLink Router.* Erişim: https://ardupilot.org/mavproxy/index.html

2. Haberleşme Protokolleri ve Standartlar

- **MAVLink Protocol Specification (v2.0).** *Micro Air Vehicle Communication Protocol.* Erişim: https://mavlink.io/en/
- **DroneKit-Python (2026).** *API for Communicating with ArduPilot via MAVLink.* Erişim: https://dronekit-python.readthedocs.io/
- **Pymavlink Library.** *Python MAVLink implementation and tools.* https://github.com/ArduPilot/pymavlink

3. Akademik ve Teknik Literatür

- **Tridgell, A. (2013).** *MAVLink and the SITL Simulator Architecture.* Open Source UAV Conference.
- **Beard, R. W., & McLain, T. W. (2012).** *Small Unmanned Aircraft: Theory and Practice.* Princeton University Press. (Aerodinamik kaldırma ve fiziksel modelleme üzerine temel kaynak).
- **Swarm Robotics & Coordination.** *Leader-Follower and Decentralized Control Models in Asymmetric Swarms.*Journal of Robotics and Automation.

4. Donanım ve Sanallaştırma Referansları

- **Apple Inc.** *Apple Silicon (M1/M2/M3) Virtualization Framework and GPU Passthrough Limitations.* [Apple Developer Documentation].
- **Canonical Ltd.** *Ubuntu 22.04 LTS Administration and Python Environment Management.*https://ubuntu.com/tutorials
- **VMware/UTM.** *Virtual Machine Hardware Acceleration and ARM-based Linux Emulation Guides.*